



Institut Africain d'Informatique
Établissement Inter-Etat d'enseignement supérieur
BP : 2263 Libreville (Gabon) Tel : (+241) 77 70 55 00 /60 43 46 00 Site
web : www.iaisiege.com E-mail : contact@iaisiege.com

RAPPORT L1 - ARCHITECTURE

Distributed Tracing avec OpenTelemetry, Grafana Tempo & Grafana

E-Service Linux - Projet Pratique 03

Rédigé par :

- ILOUMBOU Roland Junior
- ISSAMBOU Jean Samuel

Etudiants en ING2

Encadré par :

- M. ONDJIBOU

Formateur à IAI

Table des matières

1. Introduction et Contexte.....	2
1.1 Problématique.....	2
1.2 Objectifs du projet.....	2
2. Architecture Globale.....	3
2.1 Schéma de l'architecture.....	3
2.2 Description des composants.....	3
3. Architecture des Microservices.....	5
3.1 Topologie des services.....	5
3.2 Flux d'une requête de création de commande.....	5
4. Instrumentation OpenTelemetry.....	6
4.1 Principe de la propagation W3C TraceContext.....	6
4.2 Structure d'un Span instrumenté.....	6
4.3 Pipeline OpenTelemetry Collector.....	6
5. Corrélation Traces / Logs / Métriques dans Grafana.....	7
5.1 Configuration des datasources.....	7
5.2 Dashboard RED et Service Graph.....	7
6. Scénarios de Latence Implémentés.....	8
7. Déploiement sur Debian/VMware.....	9
7.1 Prérequis système.....	9
7.2 Procédure de déploiement.....	9
7.3 Commandes Docker utiles.....	9
8. Conclusion.....	10

1. Introduction et Contexte

Ce rapport présente l'architecture complète du système de tracing distribué mis en place dans le cadre du Projet 03. L'objectif est d'instrumenter une architecture microservices avec OpenTelemetry SDK et d'utiliser Grafana comme interface unifiée pour corrélérer les traces, les métriques et les logs.

1.1 Problématique

Une API e-commerce composée de 4 microservices Python/Flask souffre de latences inexplicables. L'ancien système Jaeger utilisé pour le tracing n'est pas intégré avec les outils de métriques (Prometheus) et de logs. Notre mission : migrer vers Grafana Tempo et unifier toute l'observabilité dans Grafana 10.x.

1.2 Objectifs du projet

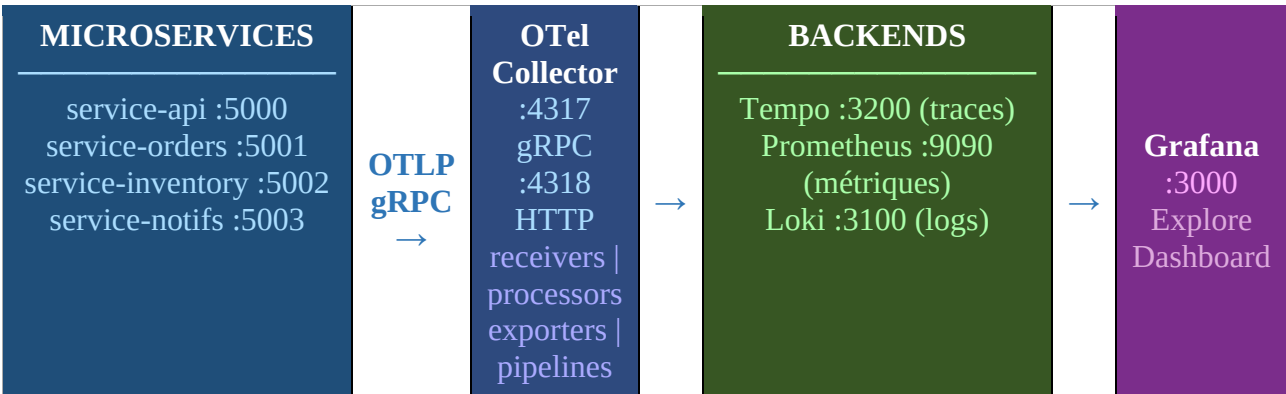
- Déployer Grafana Tempo comme backend de traces (remplacement de Jaeger)
- Instrumenter 4 microservices Python/Flask avec OpenTelemetry SDK
- Configurer la propagation de contexte W3C TraceContext entre services
- Déployer Grafana Loki + Promtail pour la collecte des logs
- Configurer les 3 datasources dans Grafana avec corrélation TraceID
- Créer un dashboard RED par service avec liens trace-to-logs
- Analyser 5 scénarios de latence via Grafana Explore

2. Architecture Globale

L'architecture repose sur trois piliers d'observabilité (les "3 piliers de l'observabilité") unifiés dans Grafana : les traces distribuées (Tempo), les métriques (Prometheus) et les logs (Loki). Tous les composants sont orchestrés via Docker Compose sur Debian/VMware.

2.1 Schéma de l'architecture

Flux de données complet (de gauche à droite) :



Flux Promtail (logs Docker) : conteneurs → Promtail :9080 → Loki :3100

2.2 Description des composants

Composant	Image Docker	Port(s)	Rôle dans l'architecture
OTel Collector	otel/opentelemetry-collector-contrib:0.95.0	4317, 4318, 8888, 8889	Point central de réception OTLP (gRPC + HTTP). Route les traces vers Tempo et les métriques vers Prometheus via pipelines configurables.
Grafana Tempo	grafana/tempo:2.4.1	3200 (HTTP), 9095 (gRPC)	Backend de stockage et requête des traces distribuées. Remplace Jaeger. Génère des métriques de service graph via le metrics_generator.
Grafana Loki	grafana/loki:2.9.4	3100	Backend de logs indexé par labels. Stocke les logs JSON des microservices avec TraceID pour corrélation trace-to-log.
Promtail	grafana/promtail:2.9.4	9080	Agent de collecte des logs Docker. Parse les logs JSON, extrait les TraceIDs via regex et les envoie à Loki.

Composant	Image Docker	Port(s)	Rôle dans l'architecture
Prometheus	prom/prometheus:v2.50.1	9090	Base de données de métriques time-series. Scrape les endpoints /metrics des microservices et reçoit les remote-write du collector. Support des exemplars pour liaison métrique → trace.
Grafana 10.x	grafana/grafana:10.4.0	3000	Interface unifiée d'observabilité. Configure les 3 datasources (Tempo, Loki, Prometheus) avec corrélation TraceID. Héberge le dashboard RED et le Service Graph.

3. Architecture des Microservices

3.1 Topologie des services

L'architecture simule une plateforme e-commerce avec 4 services interconnectés :

Service	Port	Dépendances	Responsabilités
service-api (Gateway)	5000	orders, inventory	Point d'entrée externe. Orchestre les appels aux autres services. Expose /api/orders et /api/products.
service-orders	5001	inventory, notifications	Création et suivi des commandes. Simule N+1 queries, lock contention, retry storms.
service-inventory	5002	- (feuille)	Gestion du stock. Simule full table scan (index manquant), transaction contention.
service-notifications	5003	- (feuille)	Envoi email/SMS/push. Simule timeout API externe, slow template rendering.

3.2 Flux d'une requête de création de commande

Séquence d'appels inter-services pour POST /api/orders :

- 1. Client → service-api : POST /api/orders (payload JSON)
- 2. service-api → service-inventory : POST /inventory/check (vérification disponibilité)
- 3. service-api → service-orders : POST /orders (création commande)
- 4. service-orders → service-inventory : POST /inventory/reserve (réservation stock)
- 5. service-orders → service-notifications : POST /notify (email/SMS confirmation)
- 6. service-orders → service-api → Client : réponse JSON avec order_id

Chaque appel HTTP propage le contexte W3C via l'en-tête traceparent, permettant à Tempo de reconstituer l'arbre de spans complet en un seul graphe.

4. Instrumentation OpenTelemetry

4.1 Principe de la propagation W3C TraceContext

La propagation de contexte est le mécanisme qui permet de relier des spans créés dans différents services en une trace unique. Le standard W3C TraceContext définit deux en-têtes HTTP :

- `traceparent` : version-traceid-parentid-flags (ex: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01)
- `tracestate` : liste clé=valeur pour les métadonnées spécifiques aux vendeurs

Dans notre implémentation, la bibliothèque `opentelemetry-instrumentation-requests` injecte automatiquement ces en-têtes dans chaque requête HTTP sortante. Le service récepteur extrait le contexte et crée ses spans comme enfants du span parent.

4.2 Structure d'un Span instrumenté

Chaque span contient des attributs standards OpenTelemetry (Semantic Conventions) et des attributs personnalisés :

Attribut	Valeur exemple	Description
<code>service.name</code>	<code>service-orders</code>	Identifiant du service (OpenTelemetry Resource)
<code>http.method</code>	<code>POST</code>	Méthode HTTP (convention sémantique)
<code>db.system</code>	<code>postgresql</code>	Système de base de données ciblé
<code>db.statement</code>	<code>INSERT INTO orders...</code>	Requête SQL exécutée (personnalisé)
<code>order.id</code>	<code>ord-test-0042</code>	Identifiant métier de la commande (custom)
<code>latency.scenario</code>	<code>n_plus_one</code>	Scénario de latence injecté (custom, TP)
<code>downstream.service</code>	<code>service-inventory</code>	Service appelé en aval (custom)

4.3 Pipeline OpenTelemetry Collector

L'OTel Collector est configuré avec 3 pipelines indépendants :

Pipeline	Receivers	Processors	Exporters
traces	otlp (gRPC :4317 + HTTP :4318)	memory_limiter → resource → batch	otlp/tempo → debug
metrics	otlp + prometheus (self-scrape)	memory_limiter → batch	prometheusremotewrite → prometheus exporter
logs	otlp	memory_limiter → batch	debug (Loki reçoit via Promtail)

5. Corrélation Traces / Logs / Métriques dans Grafana

5.1 Configuration des datasources

Les 3 datasources Grafana sont provisionnées automatiquement via le fichier /grafana/provisioning/datasources/datasources.yaml. Les liens de corrélation sont configurés comme suit :

Navigation	Mécanisme	Configuration Grafana
Trace → Log	TraceID dans le span Tempo → requête Loki	tracesToLogsV2 dans datasource Tempo : filtre Loki avec trace_id="\$(__trace.traceId)"
Log → Trace	Regex extraction TraceID dans les logs Loki	derivedFields dans datasource Loki : regex "trace_id":"([a-f0-9]{32})" → lien vers Tempo
Trace → Métrique	Tags du span → labels Prometheus	tracesToMetrics dans datasource Tempo : queries RED pré-définies par service.name
Métrique → Trace	Exemplars Prometheus avec trace_id	exemplarTraceIdDestinations dans datasource Prometheus → UID Tempo

5.2 Dashboard RED et Service Graph

Le dashboard RED (Rate, Errors, Duration) est organisé en 4 sections (une par service) avec pour chaque service :

- Rate : `sum(rate(flask_http_request_total[5m]))` - nombre de requêtes par seconde
- Errors : `ratio 5xx / total` - pourcentage d'erreurs
- Duration : `histogram_quantile(0.50/0.95/0.99, ...)` - latences aux percentiles
- Service Graph Panel (nodeGraph) : vue des dépendances inter-services générée par le Tempo Metrics Generator
- Panneau Logs Loki : derniers logs de tous les services avec TraceID cliquable

6. Scénarios de Latence Implémentés

5 scénarios de goulots d'étranglement sont injectés aléatoirement pour les exercices Grafana Explore :

#	Scénario	Service	Symptôme observable	Diagnostic dans Grafana
1	N+1 Queries	service-orders	N spans DB « check_item_X » en série (N = nombre d'articles)	Tempo : span « n_plus_one_queries » avec enfants multiples. Attribut antipattern=n_plus_one.
2	Lock Contention DB	service-orders	Span « lock_wait » de 300-800ms. Log WARNING avec wait_ms.	Tempo : span « db.lock_wait » avec lock.wait_ms. Loki : keyword "lock_contention".
3	Retry Storm	service-orders → notifications	3 spans « notify.retry_X » avec backoff exponentiel (100ms, 200ms, 400ms).	Tempo : spans retry avec attribut retry.attempt. Loki : keyword "retry_storm".
4	External API Timeout	service-notifications	Span « send_email/sms » de 2-4s puis fallback « queue_fallback ».	Tempo : span avec external_api.timeout=true. Loki : keyword "Timeout API".
5	Full Table Scan	service-inventory	Span « full_table_scan » de 100ms+ avec rows_scanned=50.	Tempo : span antipattern=missing_index. Event « slow_query_detected » avec recommandation.

7. Déploiement sur Debian/VMware

7.1 Prérequis système

Ressource	Minimum recommandé
RAM	4 Go (8 Go recommandé pour fluidité Grafana)
CPU	2 vCPU minimum (4 vCPU recommandé)
Disque	10 Go (images Docker + données Tempo/Loki/Prometheus)
OS	Debian 11/12 ou Ubuntu 22.04 LTS sur VMware Workstation
Docker	Docker Engine 24+ avec Docker Compose Plugin v2.20+
Réseau VM	NAT ou Bridged (accès ports 3000, 9090, 3200, 5000-5003)

7.2 Procédure de déploiement

- Étape 1 : Cloner le dépôt du projet
 - `git clone https://github.com/IRJ-Prime/Distributed_tracing.git && cd tp-tracing`
- Étape 2 : Démarrage de la stack avec le script fourni
 - `chmod +x start.sh && ./start.sh`
- Étape 3 : Génération de trafic pour produire des traces
 - `python3 -m venv venv` (# Crée l'environnement virtuel nommé 'venv' à la racine de ton projet)
 - `source venv/bin/activate` (# 3. Active l'environnement virtuel)
 - `pip3 install requests python-json-logger`
 - `python3 test_load.py --duration 300 --rate 2`
- Étape 4 : Accéder à Grafana et explorer les traces
 - `http://localhost:3000` - Login: admin / admin123
 - Menu Explore → Tempo → TraceQL: `{ duration > 500ms }`

7.3 Commandes Docker utiles

Commande	Description
<code>docker compose up -d</code>	Démarrer tous les services en arrière-plan
<code>docker compose down</code>	Arrêter et supprimer les conteneurs
<code>docker compose down -v</code>	Arrêter et supprimer les volumes (données)
<code>docker compose logs -f tempo</code>	Logs en temps réel du service Tempo
<code>docker compose ps</code>	État de tous les conteneurs
<code>docker stats</code>	Utilisation CPU/RAM des conteneurs en temps réel

Conclusion

L'architecture de tracing distribué mise en place dans ce projet réalise l'unification complète des trois piliers de l'observabilité dans une interface unique Grafana. La migration de Jaeger vers Grafana Tempo s'est accompagnée d'une intégration native avec Loki (logs) et Prometheus (métriques), rendant possible la navigation trace-to-log-to-metric sans changer d'outil.

Les 4 microservices Python/Flask instrumentés avec OpenTelemetry SDK génèrent des traces riches en attributs personnalisés, permettant d'identifier rapidement les 5 scénarios de latence simulés. La propagation W3C TraceContext garantit la continuité de la trace à travers chaque appel HTTP inter-services, quelle que soit la profondeur de la chaîne d'appels.

Ce projet constitue une base solide pour un environnement de production réel, extensible avec Grafana Pyroscope (profiling continu) pour la corrélation profil → trace, ou avec des backends de stockage objet (S3/Minio) pour Tempo à l'échelle.